EE426 / 626 Microprocessor Applications

Real-time Clock (RTC)



EE426 / 626 Microprocessor Applications

STM32F446 Real-time Clock (RTC)

BCD Calendar with subseconds, seconds, minutes, hours, day, date, month, and year

Daylight saving compensation

Two programmable alarms with interrupt support

Automatic wakeup from sleep

Reference clock detection (external precision clock)

Accurate synchronization with external clock using subsecond shift feature

Maskable interrupts from

Alarm A, Alarm B, Wakeup, Timestamp, Tamper Detection

Digital calibration circuit wth 5ppm accuracy

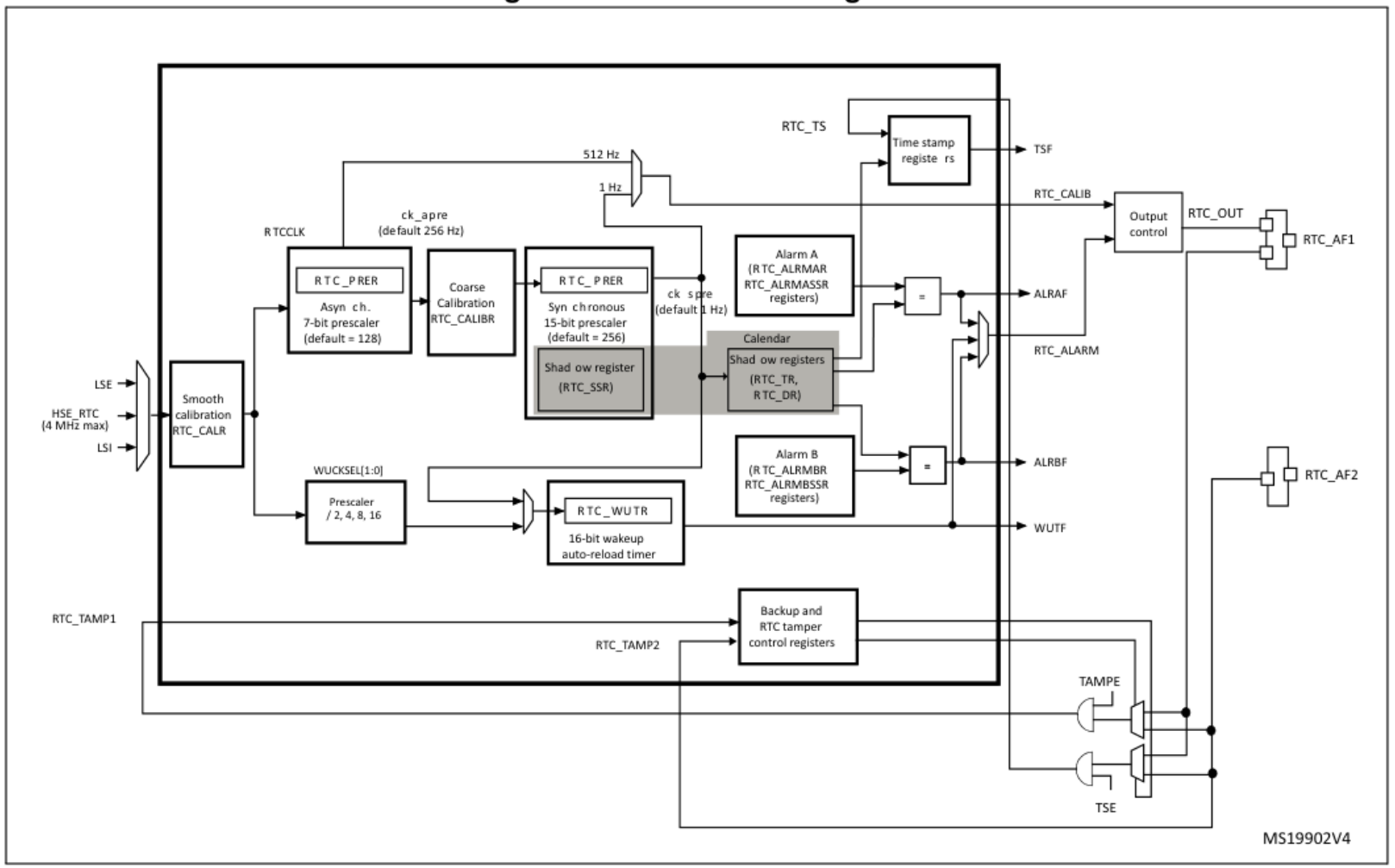
See RM0390 – STM32F446xx Advanced ARM Based 32-Bit MCUs

Chapter 22 Real-time clock (RTC)



EE426 / 626 Microprocessor Applications

Figure 240. RTC block diagram



1. On STM32F446xx devices, the RTC_AF1 and RTC_AF2 additional function are connected to PC13 and PA0, respectively.

EE426 / 626 Microprocessor Applications

STM32F446 Real-time Clock (RTC)

Step 1. Disable RTC register write protection by writing the two-value unlock key sequence to RTC_WPR.
Like the backup domain, RTC registers are protected against accidental writes, and this is a literal key sequence rather than a named bit — no CMSIS macro exists for it.

```
RTC->WPR = 0xCA;
RTC->WPR = 0x53;
```

Step 2. Enter initialization mode by setting INIT in RTC_ISR.
This pauses the calendar counter so its registers can be safely reprogrammed.

```
RTC->ISR |= RTC_ISR_INIT;                                   // (1 << 7)
```

Step 3. Poll INITF and wait until the hardware confirms initialization mode has taken effect before touching any calendar registers.

```
while (!(RTC->ISR & RTC_ISR_INITF));                        // (1 << 6)
```

Step 4. Program the asynchronous and synchronous prescalers so the 32.768 kHz LSE divides down to the 1 Hz tick the calendar needs. 128 × 256 = 32768, so PREDIV_A = 127 and PREDIV_S = 255 (divide-by-N means programming N−1).

```
RTC->PRER = (127 << 16) | (255 << 0);                       // PREDIV_A = 127, PREDIV_S = 255
```

Step 5. Select 12-hour (AM/PM) format via the FMT bit, per your requirement.

```
RTC->CR |= RTC_CR_FMT;                                      // (1 << 6)
```

EE426 / 626 Microprocessor Applications

STM32F446 Real-time Clock (RTC)

Step 5. Select 12-hour (AM/PM) format via the FMT bit

```
RTC->CR |= RTC_CR_FMT;                                    // (1 << 6)
```

Step 6. Explicitly ensure the alarms and periodic wakeup timer are disabled, along with their interrupt-enable bits.
These are already 0 at reset, but writing it explicitly documents the intent — "no alarms, nowakeup interrupts"

```
RTC->CR &= ~(RTC_CR_ALRAE | RTC_CR_ALRBE | RTC_CR_WUTE);    // ~((1<<8)|(1<<9)|(1<<10))
RTC->CR &= ~(RTC_CR_ALRAIE | RTC_CR_ALRBIE | RTC_CR_WUTIE); // ~((1<<12)|(1<<13)|(1<<14))
```

Step 7. Program the starting time into RTC_TR, BCD-packed per field.
Shown here as 12:00:00 AM (PM bit = 0, hour tens/units = 1/2 for "12"), just to give the calendar a defined starting point.

```
RTC->TR = 0x00120000;                                     // 12:00:00 AM, BCD, 12-hour format
```

Step 8. Program the starting date into RTC_DR the same way (BCD year/month/day/weekday fields)

```
RTC->DR = 0x00000000;                                     // fill in year/month/day/weekday
```

EE426 / 626 Microprocessor Applications

STM32F446 Real-time Clock (RTC)

Step 9. Exit initialization mode by clearing INIT. The calendar resumes counting from this point forward, using values just programmed.

```
RTC->ISR &= ~RTC_ISR_INIT;                                  // ~(1 << 7)
```

Step 10. Re-lock the RTC registers against accidental writes by writing any value other than the two unlock keys to RTC_WPR.

```
RTC->WPR = 0xFF;
```

Reminder: this assumes the LSE initialization sequence has already run in RCC (PWR clock enabled, backup domain unlocked, LSE on and stable, RTCSEL = LSE, RTCEN set) — without that, RTC_ISR_INITF will never assert in Step 3.

